

REST API Design

Backend Development

Resource Modeling

The first design decision in any API is identifying its resources — the nouns the API exposes (students, courses, enrollments) — and how they relate to each other. A well-modeled API mirrors the real relationships in the data: `/colleges/:collegeId/students` for students that clearly belong to a college, versus a flat `/students?collegeId=42` when the relationship is more incidental. There's no single universally 'correct' answer — the right structure depends on how the resource is actually used and queried — but consistency across the whole API matters more than any individual choice.

Status Codes & Error Handling, Applied

Building on the general status-code families, real API error handling means: validating input before doing any work and returning 400 with specific field-level errors when it fails, returning 401 vs. 403 correctly (401 means 'we don't know who you are,' 403 means 'we know who you are, but you can't do this'), and always returning errors in a consistent shape across every endpoint (e.g. `{ error: { code, message, fields } }`) so client code can handle failures generically instead of parsing a different error format per endpoint.

API Versioning

Once an API has real consumers, changing it becomes risky — a field rename or removed endpoint can break every client depending on it. Versioning strategies include putting a version in the URL (`/api/v1/students`), a custom header, or content negotiation via the Accept header. URL versioning is the most common because it's the most visible and easiest to route on the server side. The underlying discipline matters more than the specific mechanism: never make a breaking change to an existing version — add a new version instead, and give consumers time to migrate before retiring the old one.

API Documentation

An API that isn't documented is effectively unusable by anyone except the person who wrote it. Good documentation for each endpoint specifies: the HTTP method and path, required and optional parameters (with types), the shape of a successful response, and every error response that can occur. Tools like the OpenAPI/Swagger specification let this documentation be generated directly from code annotations or a schema file, keeping docs in sync with the actual implementation rather than drifting out of date as a separate hand-written document.

Learn More

GeeksforGeeks — REST API Design

<https://www.geeksforgeeks.org/rest-api-introduction/>

Swagger/OpenAPI Documentation

<https://swagger.io/docs/>

MDN Web Docs — HTTP Response Status Codes

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>